

Privora

Docker Deployment Guide

Prepared for: Bassam — Founder, Privora
April 2026 | Confidential

Privora — Docker Deployment Guide

Overview

This guide covers running Privora using Docker and Docker Compose for both local development and production on-premises deployment.

The backend already includes a production-ready `Dockerfile`. This guide adds the full `docker-compose.yml` for the complete stack.

Files

```
Bassam/  
  backend/  
    Dockerfile          ← Already exists (production backend)  
  frontend/  
    Dockerfile          ← Frontend build + Nginx  
    docker-compose.yml  ← Full stack orchestration  
    docker-compose.dev.yml ← Development override  
    .env.docker         ← Docker environment variables template
```

1. Backend Dockerfile (Existing)

Located at `backend/Dockerfile`:

```

FROM node:20-alpine

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm ci --only=production

# Copy source
COPY . .

# Logs directory
RUN mkdir -p logs

# Non-root user for security
RUN addgroup -S privora && adduser -S privora -G privora
RUN chown -R privora:privora /app
USER privora

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --
retries=3 \
  CMD wget -qO- http://localhost:5000/api/v1/health || exit 1

CMD ["node", "src/server.js"]

```

2. Frontend Dockerfile

Create `frontend/Dockerfile`:

```

FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL
ARG VITE_DEMO_MODE=false
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Create `frontend/nginx.conf`:

```

server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # React Router support – redirect all routes to index.html
    location / {
        try_files $uri $uri/ /index.html;
    }
}

```

```

# Cache static assets
location ~* \.(js|css|png|jpg|gif|ico|svg|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}

# Security headers
add_header X-Frame-Options "SAMEORIGIN";
add_header X-Content-Type-Options "nosniff";
add_header X-XSS-Protection "1; mode=block";
}

```

3. Docker Compose — Production

Create `docker-compose.yml` at the project root:

```

version: '3.9'

services:
  db:
    image: postgres:15-alpine
    container_name: privora_db
    restart: unless-stopped
    environment:
      POSTGRES_DB: privora
      POSTGRES_USER: privora_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432" # Only accessible from localhost
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U privora_user -d privora"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - privora_internal

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: privora_backend
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      NODE_ENV: production
      PORT: 5000
      DB_URL: postgresql://privora_user:${DB_PASSWORD}@db:5432/privora
      DB_DIALECT: postgres
      JWT_SECRET: ${JWT_SECRET}
      JWT_REFRESH_SECRET: ${JWT_REFRESH_SECRET}
      JWT_EXPIRES_IN: 15m
      JWT_REFRESH_EXPIRES_IN: 7d

```

```

    OTP_SECRET: ${OTP_SECRET}
    SMTP_HOST: ${SMTP_HOST}
    SMTP_PORT: ${SMTP_PORT}
    SMTP_USER: ${SMTP_USER}
    SMTP_PASS: ${SMTP_PASS}
    EMAIL_FROM: ${EMAIL_FROM}
    CLIENT_URL: ${CLIENT_URL}
  ports:
    - "127.0.0.1:5000:5000"    # Only accessible from localhost
  (Nginx proxies)
  volumes:
    - backend_logs:/app/logs
  networks:
    - privora_internal
    - privora_external

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  args:
    VITE_API_URL: ${VITE_API_URL}
    VITE_DEMO_MODE: "false"
  container_name: privora_frontend
  restart: unless-stopped
  depends_on:
    - backend
  ports:
    - "80:80"
    - "443:443"
  networks:
    - privora_external

volumes:
  postgres_data:
  backend_logs:

networks:
  privora_internal:
    internal: true    # DB not accessible from outside
  privora_external:
    driver: bridge

```

4. Docker Compose — Development

Create `docker-compose.dev.yml`:

```

version: '3.9'

services:
  db:
    ports:
      - "5432:5432"    # Expose DB in development for direct
                        access

  backend:
    build:
      context: ./backend
      target: development
    volumes:
      - ./backend/src:/app/src    # Hot reload source files

```

```
environment:
  NODE_ENV: development
  command: npm run dev

frontend:
  build: ./frontend
  volumes:
    - ./frontend/src:/app/src
  ports:
    - "5173:5173"
  command: npm run dev -- --host
```

5. Environment Variables for Docker

Create `.env.docker` (copy and fill in values — never commit this file):

```
# Database
DB_PASSWORD=strong_random_password_here

# JWT (generate: openssl rand -hex 64)
JWT_SECRET=your_64_char_jwt_secret
JWT_REFRESH_SECRET=your_64_char_refresh_secret

# OTP
OTP_SECRET=your_otp_secret

# Email
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=your_sendgrid_key
EMAIL_FROM=no-reply@privora.sa

# URLs
CLIENT_URL=https://app.privora.sa
VITE_API_URL=https://api.privora.sa/api/v1
```

6. Running the Stack

Build and Start (Production)

```
# Copy and configure env file
cp .env.docker .env
nano .env # Fill in all values

# Build all images
docker compose build

# Start all services in background
docker compose up -d

# Check status
docker compose ps

# View logs
```

```
docker compose logs -f backend
docker compose logs -f db
```

Start (Development)

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml
up
```

Stop

```
docker compose down
```

Stop and Delete Data

```
docker compose down -v # WARNING: deletes all database data
```

7. Database Operations in Docker

Run Migrations

```
docker compose exec backend npm run db:migrate
```

Access PostgreSQL Shell

```
docker compose exec db psql -U privora_user -d privora
```

Backup Database

```
docker compose exec db pg_dump -U privora_user privora > backup-
$(date +%Y%m%d).sql
```

Restore Database

```
cat backup-20260413.sql | docker compose exec -T db psql -U
privora_user -d privora
```

8. Production SSL with Cloudflare

The recommended setup for production:

User → Cloudflare (SSL/WAF) → Your Server (Docker stack)

Cloudflare handles SSL termination. Your server runs Nginx on port 80.

Cloudflare proxies HTTPS and passes traffic to your server as HTTP internally.

In this setup, your `docker-compose.yml` frontend service exposes only port 80.

Cloudflare handles the HTTPS layer.

9. Updating the Application

```
# Pull latest code
git pull origin main

# Rebuild and restart only changed services
docker compose build backend frontend
docker compose up -d backend frontend

# Verify health
docker compose ps
curl http://localhost:5000/api/v1/health
```

10. Resource Monitoring

```
# Live resource usage
docker stats

# Disk usage by volume
docker system df -v

# Clean unused images
docker image prune -f
```

Quick Reference

Command	Action
<code>docker compose up -d</code>	Start all services
<code>docker compose down</code>	Stop all services
<code>docker compose ps</code>	Service status
<code>docker compose logs -f backend</code>	Stream backend logs
<code>docker compose exec backend sh</code>	Shell inside backend container

Command	Action
<code>docker compose build</code>	Rebuild all images
<code>docker compose restart backend</code>	Restart one service